Task 2: Introduction to Web Application Security

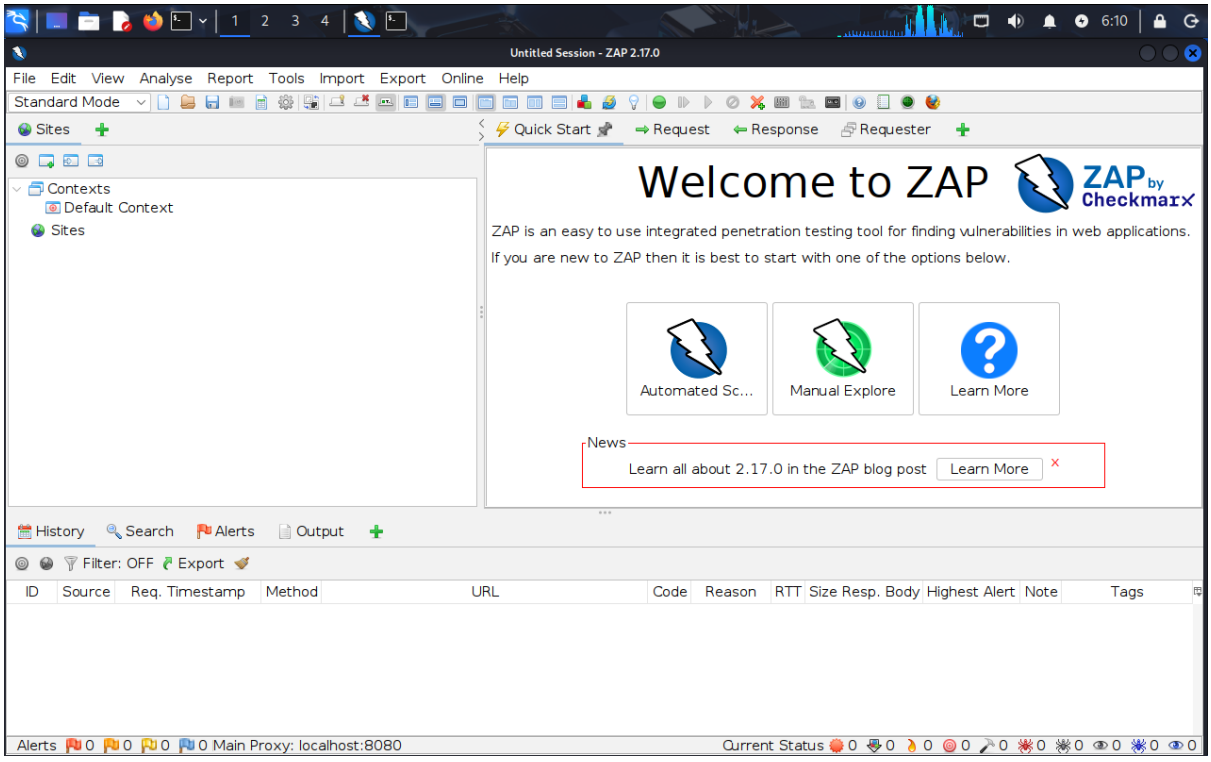
1. Installing OWASP ZAP

- OWASP ZAP (Zed Attack Proxy) is an open-source web application security scanner used to identify vulnerabilities in web applications. It helps security professionals test applications for common security issues such as SQL Injection, Cross-Site Scripting (XSS), and misconfigured security headers.
- In this task, OWASP ZAP was installed in Kali Linux using the terminal command:
- sudo apt install zaproxy
- After installation, the tool was successfully configured for scanning and analyzing web applications.

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install zaproxy
zaproxy is already the newest version (2.17.0-0kali1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1697
```

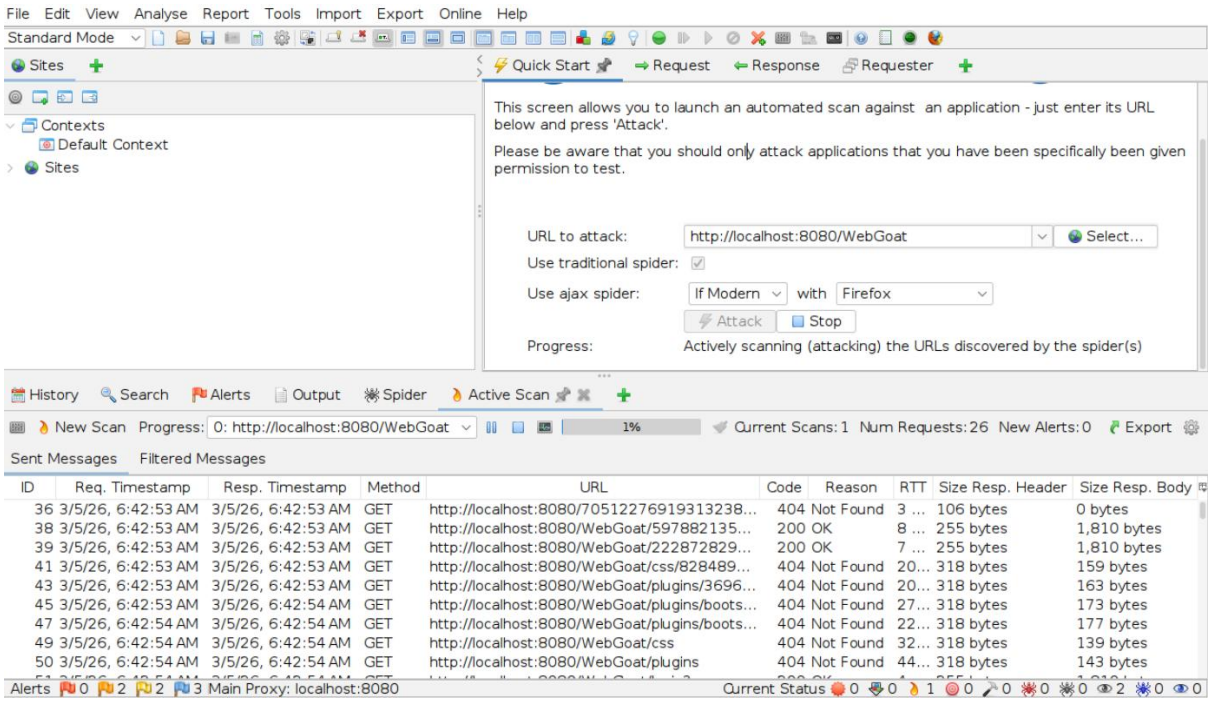
2. Launching OWASP ZAP

- After installation, OWASP ZAP was launched in Kali Linux to perform a security analysis of a vulnerable web application called WebGoat.
- OWASP ZAP acts as a proxy between the browser and the web application, allowing it to monitor traffic, detect vulnerabilities, and perform automated scans.



3. Running WebGoat

- WebGoat is a deliberately vulnerable web application created by OWASP to help developers and security professionals learn about web application security issues.
- The WebGoat application was started using the terminal in Kali Linux. After successful execution, the application was accessible in the browser at:
- http://localhost:8080/WebGoat
- This allowed the application to be scanned and analyzed using OWASP ZAP.
- OWASP ZAP performing an automated vulnerability scan on the WebGoat web application.



4. Vulnerability Scanning

An automated vulnerability scan was performed using OWASP ZAP against the WebGoat application.

The scanner analyzed the web application and detected multiple security vulnerabilities including:

- SQL Injection
- Missing Anti-CSRF Tokens
- Content Security Policy (CSP) Header Not Set
- Cookie Security Issues
- Session Management Weaknesses

These vulnerabilities highlight common security weaknesses that can exist in web applications if proper security practices are not followed.

1. SQL Injection

Description

SQL Injection is a web security vulnerability that allows attackers to manipulate the queries that an application makes to its database. By inserting malicious SQL code into input fields, attackers can change the logic of database queries.

Discovery

The vulnerability was discovered during the automated vulnerability scan performed using OWASP ZAP on the WebGoat web application.
OWASP ZAP detected a potential SQL Injection vulnerability in the following URL:

http://localhost:8080/WebGoat/register.mvc

The scanner identified that the agree parameter could be manipulated using a malicious SQL payload.

Example Payload

agree' OR '1'='1' --

Why It Is Dangerous

If exploited, SQL Injection can allow attackers to:

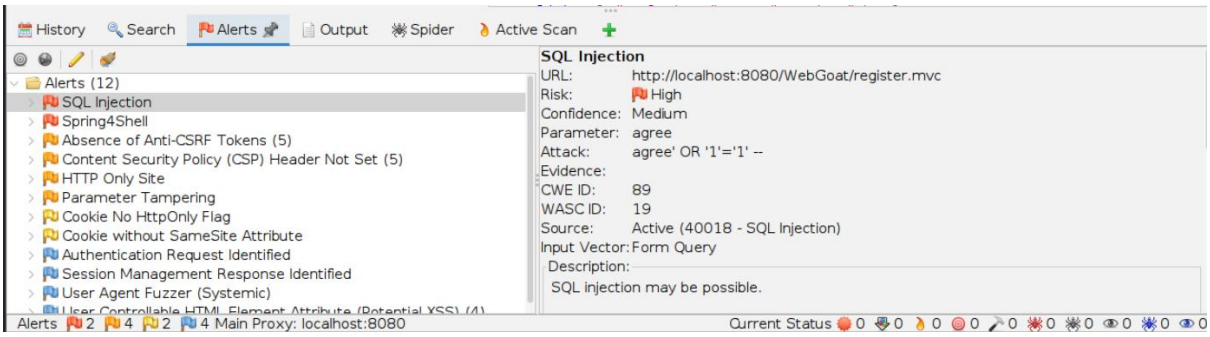
- Bypass authentication mechanisms
- Access sensitive database information
- Modify or delete database records
- Gain unauthorized access to the system

This vulnerability can lead to serious data breaches and compromise the entire application.

Exploitation Process

The attacker inputs a malicious SQL query into an input field. The application executes the injected query along with the original query, allowing the attacker to manipulate the database logic.

Screenshot



Mitigation

To prevent SQL Injection vulnerabilities:

- Use prepared statements or parameterized queries
- Validate and sanitize all user inputs
- Implement proper input filtering
- Use ORM frameworks that prevent raw SQL queries
- Apply least privilege access to database accounts

---

2. Absence of Anti-CSRF Tokens

Description

Cross-Site Request Forgery (CSRF) occurs when an attacker tricks a user into performing actions on a web application without their knowledge.

Discovery

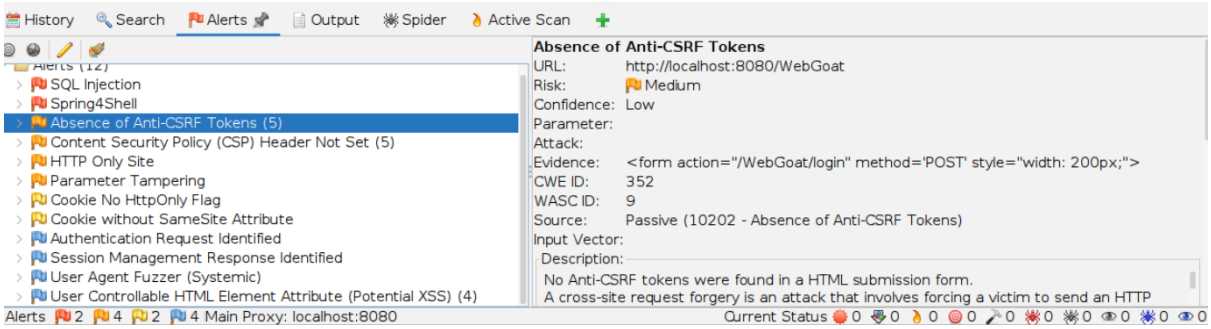
OWASP ZAP detected that the WebGoat application does not use Anti-CSRF tokens in certain forms. This means requests can be forged without proper verification.

Why It Is Dangerous

An attacker could create a malicious link or webpage that performs actions on behalf of a logged-in user, such as:

- Changing account settings
- Performing financial transactions
- Submitting unauthorized requests

Screenshot



Mitigation

To prevent CSRF attacks:

- Implement CSRF tokens in forms
- Validate request origin using SameSite cookies
- Use secure authentication mechanisms
- Validate requests on the server side

---

3. Content Security Policy (CSP) Header Not Set

Description

A Content Security Policy (CSP) header helps protect web applications against Cross-Site Scripting (XSS) and other injection attacks.

Discovery

OWASP ZAP detected that the web application does not include a Content Security Policy header in HTTP responses.

Why It Is Dangerous

Without CSP protection, attackers may inject malicious scripts that can:

- Steal session cookies
- Execute unauthorized actions
- Manipulate webpage content

Mitigation

To prevent this issue:

- Implement a Content Security Policy header
- Restrict allowed script sources
- Disable inline scripts when possible
- Monitor policy violations

Example CSP header:

Content-Security-Policy: default-src 'self';

---

4. Cookie Without HttpOnly Flag

Description

Cookies without the HttpOnly flag can be accessed through JavaScript.

Discovery

OWASP ZAP detected cookies that do not have the HttpOnly security attribute enabled.

Why It Is Dangerous

If attackers perform an XSS attack, they can steal session cookies and hijack user sessions.

Mitigation

To prevent this issue:

- Set the HttpOnly flag on cookies

- Use Secure cookies over HTTPS
- Implement proper session management

Example:

Set-Cookie: sessionid=12345; HttpOnly; Secure;

Conclusion

The vulnerability assessment performed using OWASP ZAP identified several security issues within the WebGoat application. These vulnerabilities demonstrate common weaknesses in web applications and highlight the importance of implementing secure coding practices and regular security testing to protect sensitive data and system integrity.